Salut! Si tu regardes ça j'imagine que mon application t'intéresse, bref ne perdons pas plus de temps;
PS: l'application marche sur téléphone et pc (merci streamlit), mais pour l'ouvrir sur téléphone il faut qu'un pc la fasse tourner (un peu comme un serveur) et les deux appareils doivent connecté au même wi-fi, quand tu lances l'application, dans le terminal tu as une ligne "network url" c'est ce lien qu'il faut utiliser pour se connecter depuis un autre appareils.

Installation et Lancement

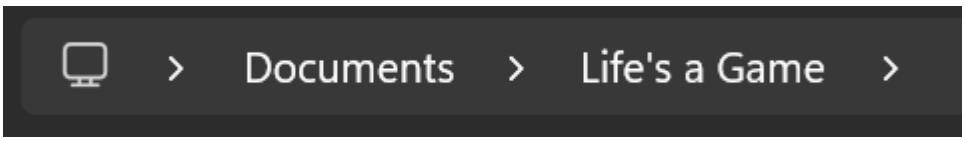
Comment on installe un programme: déjà vous devez ouvrir Github (j'ai mis le lien du projet dans la description), ensuite, dans la page du projet, vous verrez plusieurs dossier, ils ne sont pas tous intéressant si vous lisez ceci, ce qu'on veut nous c'est simplement les fichiers Python, les png des rangs et le dossier .streamlit (optionnel celui là, c'est simplement pour personnaliser les couleurs de l'app).

Sur votre pc faites bien attention à avoir votre dossier organiser comme celui-ci

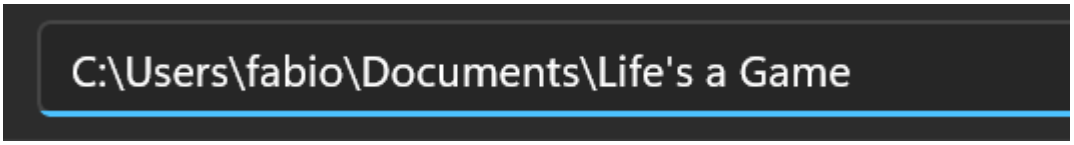
.streamlit	28/03/2026 23:14	Dossier de fichiers	
__pycache__	03/04/2026 22:29	Dossier de fichiers	
data	02/04/2026 15:45	Dossier de fichiers	
sprite	03/04/2026 10:58	Dossier de fichiers	
accueil	03/04/2026 21:52	Fichier source Python	5 Ko
arbre	01/04/2026 11:50	Fichier source Python	5 Ko
bureaux	30/03/2026 17:52	Fichier source Python	14 Ko
journal	27/03/2026 07:21	Fichier source Python	4 Ko
main	03/04/2026 21:20	Fichier source Python	4 Ko
oracle	31/03/2026 10:17	Fichier source Python	5 Ko
settings	01/04/2026 11:30	Fichier source Python	2 Ko
statistiques	17/03/2026 09:13	Fichier source Python	1 Ko
utils	03/04/2026 22:29	Fichier source Python	13 Ko
utils_ai	30/03/2026 22:53	Fichier source Python	6 Ko
utils_asset	03/04/2026 22:17	Fichier source Python	3 Ko

Ce qui est important c'est de ne pas mettre des fichiers dans d'autres dossiers (ou alors il faut modifier le code, parce qu'il est réglé pour aller chercher tel fichier dans tel dossier, donc si on change l'organisation il faut lui indiquer le nouveau chemin).

Ensuite vous allez ouvrir un terminal, aller dans le dossier du projet (dans le terminal: cd + le chemin vers le dossier en question par exemple: cd C:\Users\user1\Documents\Life's a Game)



puis double clic dessus et le chemin apparaît



Ensuite ici il faut installer les dépendances du projet (les bibliothèque que j'ai utilisé) et pour ça rien de très compliqué: (*)

- il suffit de taper dans le terminal pip install -r requirements.txt (si tu as ins ou alors tu installe à la main toutes les bibliothèques qui sont dans requirement.txt avec pip install (dans le terminal) et le nom de la bibliothèque, ex: pip install streamlit

Puis une fois que vous avez tout installer, créer et activer votre environnement virtuel il y a deux possibilitées:

- Si vous avez intégrer Python au PATH de votre pc il suffit de taper dans le terminal streamlit run main.py
- Si votre Python n'est pas intégrer au PATH il suffit d'écrire dans le terminal python -m streamlit run main.py

Maintenant le programme tourne tant que le terminal reste ouvert (ou pour le stopper sans fermer le terminal il suffit de faire CTRL C), vos modifications sont sauvegardées automatiquement dans des fichier .csv dans le dossier du programme, vous pouvez les modifiers à la main aussi (le nom d'une quête ou d'un domaine, l'xp d'une quête ou d'un domaine etc...)

*quand on créé ou installe un programme il est recommandé d'utiliser un environnement virtuel, j'ai pas encore compris à 100% le fonctionnement donc je ne vais pas m'étaler, si vous voulez en savoir plus encore une fois Graven à fait une vidéo ou il expliquera beaucoup mieux que moi.

Explication rapide du fonctionnement

Déjà l'application est codée en Python, j'ai utilisé streamlit (bibliothèque) pour l'interface, cette bibliothèque est super documentée donc si vous voulez ajoutez d'autre fonctionnalités dont je ne parle pas ici vous devriez trouver facilement des solutions en ligne.

L'app est divisée en plusieurs page, pour lancer le programme on run main.py, ensuite il y a 2 autre catégorie de fichier python (dans mon app): les **utils**, dans lesquels j'ai regroupé toutes les différentes fonctions, c'est de l'optimisation, ça évite de charger toutes les fonctions à chaque fois qu'on lance une page, à la place on importe, depuis la page utils en question, seulement les fonctions dont on a besoin pour une page (par exemple la fonction pour trouver le niveau du joueur connecté etc..);

Et les "pages" comme accueil.py etc, ça c'est les différentes pages de l'application, la navigation entre elles est gérée main.py (donc vous pouvez inverser, changer le nom ou même supprimer des pages super facilement).

Le thème de l'application est modifiable dans config.toml (dans le dossier .streamlit) je ne me suis pas trop renseigné sur cette partie ci donc je ne vais pas m'étaler, globalement ce qui nous

intéresse c'est ces lignes ci pour le thème principal

```
[theme]
base = "dark"
primaryColor = "#b90f0f"
backgroundColor = "#2d2020"
secondaryBackgroundColor = "#130c0c"
codeBackgroundColor = "#1f1b1b"
textColor = "#ffffff"
linkColor = "#188b82"
borderColor = "#727272"
```

et celles ci pour la barre latérale ,

```
[theme.sidebar]
backgroundColor = "#010101"
secondaryBackgroundColor = "#4f0808"
codeBackgroundColor = "#2f2c2c"
borderColor = "#727272"
```

et si vous avez un doute vous pouvez toujours faire quelques recherches.

Petit détail supplémentaire à propos du thème: les couleurs des diagrammes (la courbe de progression et le diagramme radial) sont modifiables dans leur pages (bureau.py et accueil.py), pas dans le fichier config.toml.

Les Mécaniques de Maîtrise

Pour que l'expérience soit équilibrée, j'ai mis en place quelques règles mathématiques :

1. Calcul du Niveau

Le niveau n'est pas linéaire (exponentiel, de plus en plus dur). J'utilise une formule de type :

$$Niveau = \sqrt{\frac{XP}{100}}$$

(donc pour atteindre le niveau 100 dans un sous domaine il faut 1.000.000 d'xp

- **Modif :** Si tu veux que la progression soit plus rapide ou plus lente, tu peux modifier la fonction get_level (dans utils.py)

2. La perte d'xp (Perte de Fraîcheur)

Si tu n'exerces pas une compétence, elle "s'efface" visuellement (ton score de fraîcheur baisse).

- **Fonctionnement :** L'app calcule le nombre de jours entre aujourd'hui et ta dernière activité.

- **Règle :** Après **7 jours** d'inactivité, tu perds environ **4%** de fraîcheur par jour.
- **Impact :** L'Oracle utilisera cette donnée pour te gronder ou te motiver.

3. XP Récursive

C'est une particularité du projet : si tu gagnes de l'XP en "Boxe", cette XP remonte automatiquement à "Sport" et à ton niveau "Global". Tu peux créer autant de sous-catégories que tu veux, la somme sera toujours juste.

Graphismes et Données

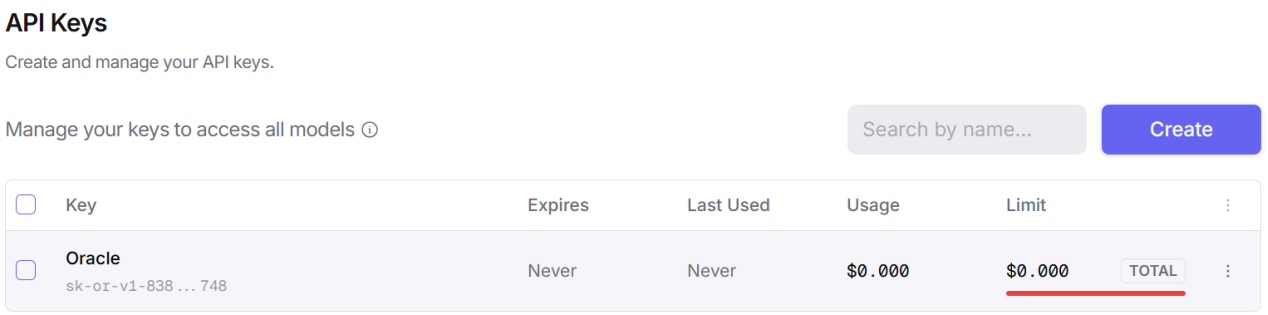
Graphismes: j'ai mis mes graphismes de niveau dans le dossier "sprite", vous pouvez changer facilement les icônes de rang ducoup, mais il faudra ajuster les masques (dans utils_asset)

Données: Tout est stocké dans des fichiers .csv (data_user.csv et quests_user.csv) qui sont normalement rangés dans le dossier data.

Tu peux les ouvrir avec Excel ou Notepad pour corriger une erreur de saisie manuellement; le fichier user.csv permet à l'application de savoir qui est connecté (c'est là qu'on garde les noms d'utilisateurs et les "mots de passes")

IA: Clé API (important)

Dans le code à télécharger j'ai laisser le champs de la clé vide (ou avec une indication); pour que ça fonctionne vous devez vous créer une clé sur **openrouter**, pour ça rien de très compliqué: il faut se créer un compte puis créer une clé



comme je ne compte pas utiliser de modèle payant j'ai mis une limite de budget à 0$, mais dans tous les cas notre fonction ne cherche que des modèles gratuits, et à moins que vous ayez mis quelque dollars sur votre compte **openrouter**, il n'utilisera jamais un modèle payant (parce que pas de sous), une fois que vous avez créé votre clé (faites bien attention à ne pas donner votre clé, surtout si vous n'avez pas mis de limite de budget, sinon n'importe qui pourrait l'utiliser; si ça arrive

vous pouvez toujours supprimer la clé et en créer une nouvelles) puis une fois que vous avez votre clé il suffit de la copier à sa place

```
def appel_oracle_securise(system_prompt, user_prompt):
    client = openai.OpenAI(
        base_url="https://openrouter.ai/api/v1",
        api_key="METTRE CLÉ API ICI (ENTRE LES APOSTROPHES)",
    )
```

Un dernier conseil : L'application est faite pour évoluer. N'hésite pas à fouiller dans utils.py, c'est là que tu trouveras tous les leviers pour ajuster la difficulté de ton propre RPG !

Et si après tout ça tu as encore des questions, des retours ou même des idées tu m'en parler, sur mon insta (le lien est dans lien est dans la description de la chaîne) ou même en commentaires!